



Dart

Day 2

NUM

num Data Type

num is the parent type of both int and double.



```
num value = 10;  
  
print(value);  
print(value.runtimeType);  
  
value = 10.5;  
  
print(value);  
print(value.runtimeType);
```



NULL
SAFETY

What is Null?

Without Null Safety, the program may crash when using a null value.

Why do we need Null Safety?

Null means that the variable has no value.

Null Safety Operators

1. Null Assertion Operator (!)

I am sure this value is not null.

⚠ If it is null → Runtime Error



```
print("please enter your name");  
String name = stdin.readLineSync()!;  
print("Hello $name");
```

2. Nullable Operator (?)

The variable can store a value or null



```
String? name;
```

Null Safety Operators

3. Null-aware Operator (?.)

If variable is null return null and doesn't execute function / property so app doesn't crash



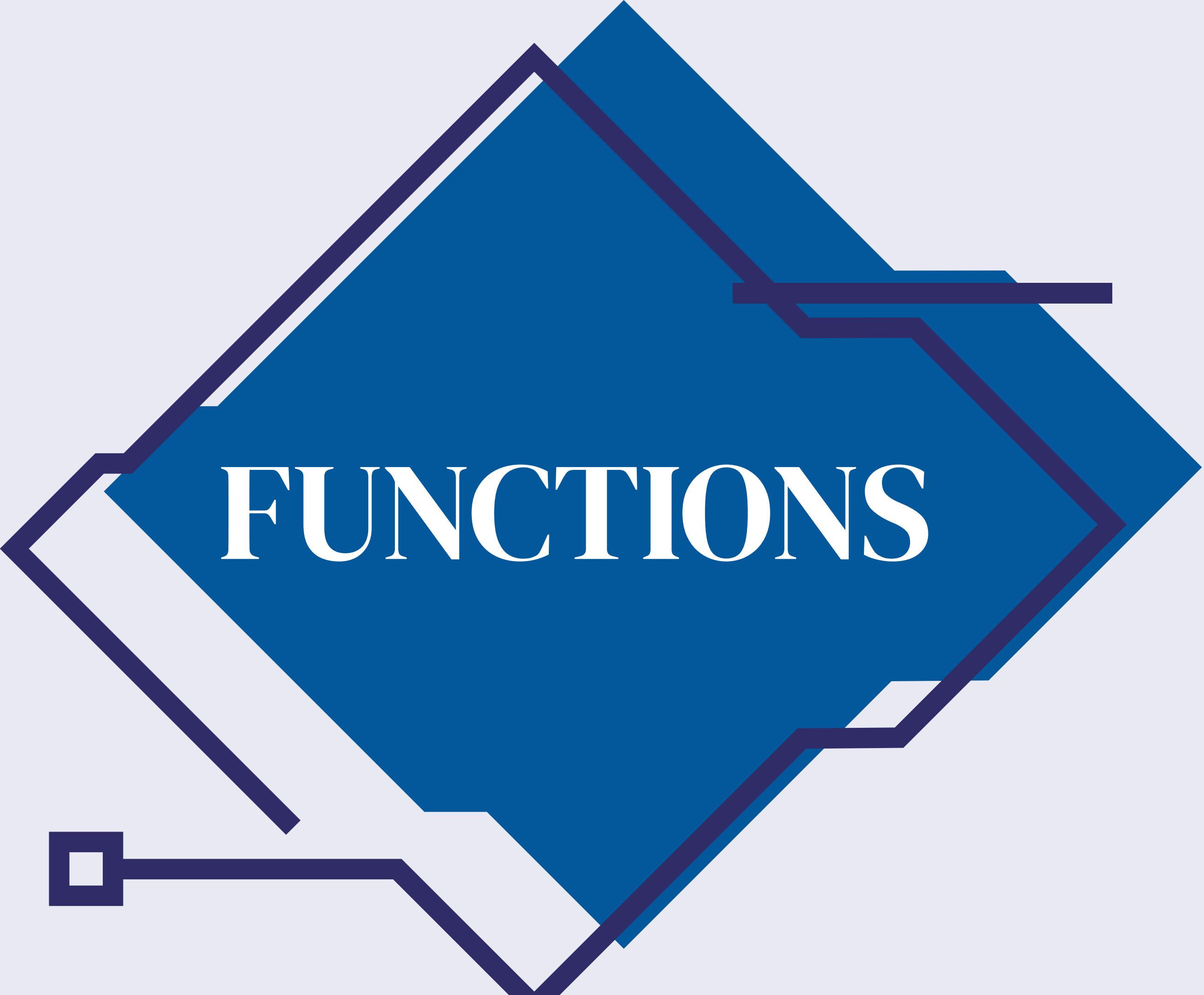
```
String? name;  
print(name?.length);
```

4. Default Value Operator

If the variable is null, use the default value.



```
String? city;  
print(city ?? "Cairo");
```

The image features a large, solid blue diamond shape centered on a light blue background. Overlaid on this diamond and extending into the background are several dark blue, thick lines that form a complex, angular, and somewhat circuit-like pattern. These lines intersect and branch out, creating a sense of movement and structure. The word "FUNCTIONS" is written in a white, serif, all-caps font, centered horizontally across the middle of the blue diamond.

FUNCTIONS

Why Functions

Avoid repetition – write the code once, use it as many times as you need

Organize code – break a big program into smaller, manageable pieces

Easier to maintain – fix a bug in one place instead of everywhere it's repeated

Reusability – call the same function from different parts of your program



Function Body

The function body is everything inside the curly braces { }

It contains the actual logic/instructions that execute when the function is called

Runs every time the function is called, from top to bottom

Parameters

Think of a Function as a machine, and a Parameter is the "slot" where you feed input for the machine to work with.

Parameter = the name agreed on inside the function's definition (like an empty reserved slot)

Argument = the actual value you pass when calling the function (what fills that slot)



```
returnType functionName(parameters) {  
    // function body - the code that runs  
    // when the function is called  
    return value;  
}
```

Return Keyword & Function Data Type

`return` – sends a value back to wherever the function was called from, and stops the function immediately

The return type (written before the function name) tells Dart what kind of value the function will give back — e.g., `int`, `String`, `bool`

`void` – used when the function doesn't return anything at all The value after `return` must match the declared return type



```
int add(int a, int b) {  
    return a + b; // sends the result back  
    to the caller  
}  
  
void main() {  
    int result = add(5, 3);  
    print(result); // 8  
}
```

01 No Parameter + No Return void

A function that does not receive any parameters and does not return a value.



```
void greet() {  
    print("Hello");  
}
```

02 Parameter + No Return void

A function that receives parameters but does not return a value.



```
void greet(String name) {  
    print("Hello $name");  
}
```

03 No Parameter + Return return

A function that does not receive parameters but returns a value.



```
String getName(){  
    return name ; //return object name  
    variable value  
}
```

04 Parameter + Return return

A function that receives parameters and returns a value.



```
double recArea( double width, double  
height ){  
    double area = width*height;  
    return area;  
}
```

05 Named Function name

A regular function declared with a specific name that can be called multiple times.



```
String getName(){  
    return name ; //return object name  
    variable value  
}
```

06 Anonymous Function

A function without a name, defined directly where it's needed

Commonly passed as an argument to another function (like `.forEach()`, `.map()`)

Can be stored in a variable and called later



```
friends.forEach((name) {  
    print('Hello $name');  
});
```



```
void main() {  
    Function discountRule = (double price)  
=> price; // no discount  
    print('Normal: ${discountRule(100)}');  
    // 100  
  
    // vip-customer  
    discountRule = (double price) => price  
    * 0.8; // discount 20%  
  
    print('VIP: ${discountRule(100)}'); //  
    80  
}
```

07 Arrow Function (Fat Arrow) =>

A shorthand syntax written using the => operator for single-expression functions.

=> replaces { return ... ; } — no need to write return or curly braces

Can only contain one expression, not multiple statements



```
double recArea( double width, double  
height )=> width*height;
```



```
// Regular function  
int add(int a, int b) {  
    return a + b;  
}  
  
// Arrow function - shorthand for a  
single expression  
int add2(int a, int b) => a + b;  
  
void main() {  
    print(add2(5, 3)); // 8
```

08 Lambda Function inline

A compact function expression written inline.



```
var multiply = (int a, int b) => a*b;
```

09 Higher Order Function

A function that receives another function as a parameter or returns a function.

10 Callback Function

A function passed to another function to be executed later.



```
int applyOperation(int a, int b, int  
Function(int, int) operation) {  
    return operation(a, b);  
}  
  
int add(int x, int y) {  
    return x + y;  
}  
  
int multiply(int x, int y) {  
    return x * y;  
}  
  
void main() {  
    print(applyOperation(5, 3, add));  
    // 8  
    print(applyOperation(5, 3, multiply));  
    // 15  
}
```

11 Recursive Function

A function that calls itself repeatedly until a stopping condition is met.



```
int factorial(int number){  
    if(number==1) return 1;  
    return number * factorial(number-1);  
}
```

12 Async Function

A function that performs asynchronous operations using async and await.



```
Future<void> getData() async {  
    print("Fetching data...");  
  
    await Future.delayed(Duration(seconds:  
3));  
  
    print("Data loaded successfully");  
}
```

13 Sync Function

A function that executes statements sequentially in a synchronous manner.



```
void displayNumbers() {  
    print(1);  
    print(2);  
    print(3);  
}
```

14 Generator Function

A function that generates a sequence of values using sync* or async*.



```
Iterable<int> numbers() sync* {  
    yield 1;  
    yield 2;  
    yield 3;  
}
```

15 Local Function (Nested)

A function declared inside another function.



```
void main() {  
    void sayHi() {  
        print('Hi');  
    }  
  
    sayHi();  
}
```

16 Function as Variable var

A function stored inside a variable and used like a normal value.



```
void main() {  
    var greeting = () {  
        print("Hello");  
    };  
    greeting();  
}
```

17 Typedef Function

A function declared inside another function.



```
void main() {  
    void sayHi() {  
        print('Hi');  
    }  
  
    sayHi();  
}
```

18 Optional Positional Parameter []

A function containing optional parameters placed inside square brackets [].



```
String Welcoming([String ? name ]){  
    return "Hello $name";  
}
```

19 Default Parameter default

A function that assigns default values to parameters if no value is provided.



```
String Welcoming([String ? name = ""]){  
    return "Hello $name";  
}
```

20 Optional Named Parameter { }

A function containing optional named parameters placed inside curly braces {}.



```
String Welcomming({String ? name ,  
String ? lastName}){  
    return "Hello $name $lastName";  
}
```

21 Required Named Parameter

A function with named parameters marked as required.



```
String Welcomming({required String name,  
required String lastName}) {  
    return "Hello $name $lastName";  
}
```

Function Scope

Scope = where a variable can be seen and used

Variables declared inside a function are local
— they only exist inside that function

Variables declared outside any function are
global — accessible everywhere

A local variable disappears once the
function finishes running



```
int x = 10; // global scope - accessible everywhere
```

```
void myFunction() {  
    int y = 5; // local scope - only accessible inside myFunction  
    print(x); // ✓ works, x is global  
    print(y); // ✓ works, y is local to this function  
}
```

```
void main() {  
    myFunction();  
    print(x); // ✓ works  
    print(y); // ✗ Error! y doesn't exist outside myFunction  
}
```

Hands-On Task (5 minutes)

Task 1 — Arrow Function Challenge



```
int calculateDiscount(int price, int  
discountPercent) {  
    return price - (price *  
discountPercent ~/ 100);  
}
```

calculateDiscount(200, 20)



Hands-On Task (5 minutes)

Task 2 — Scope Trap

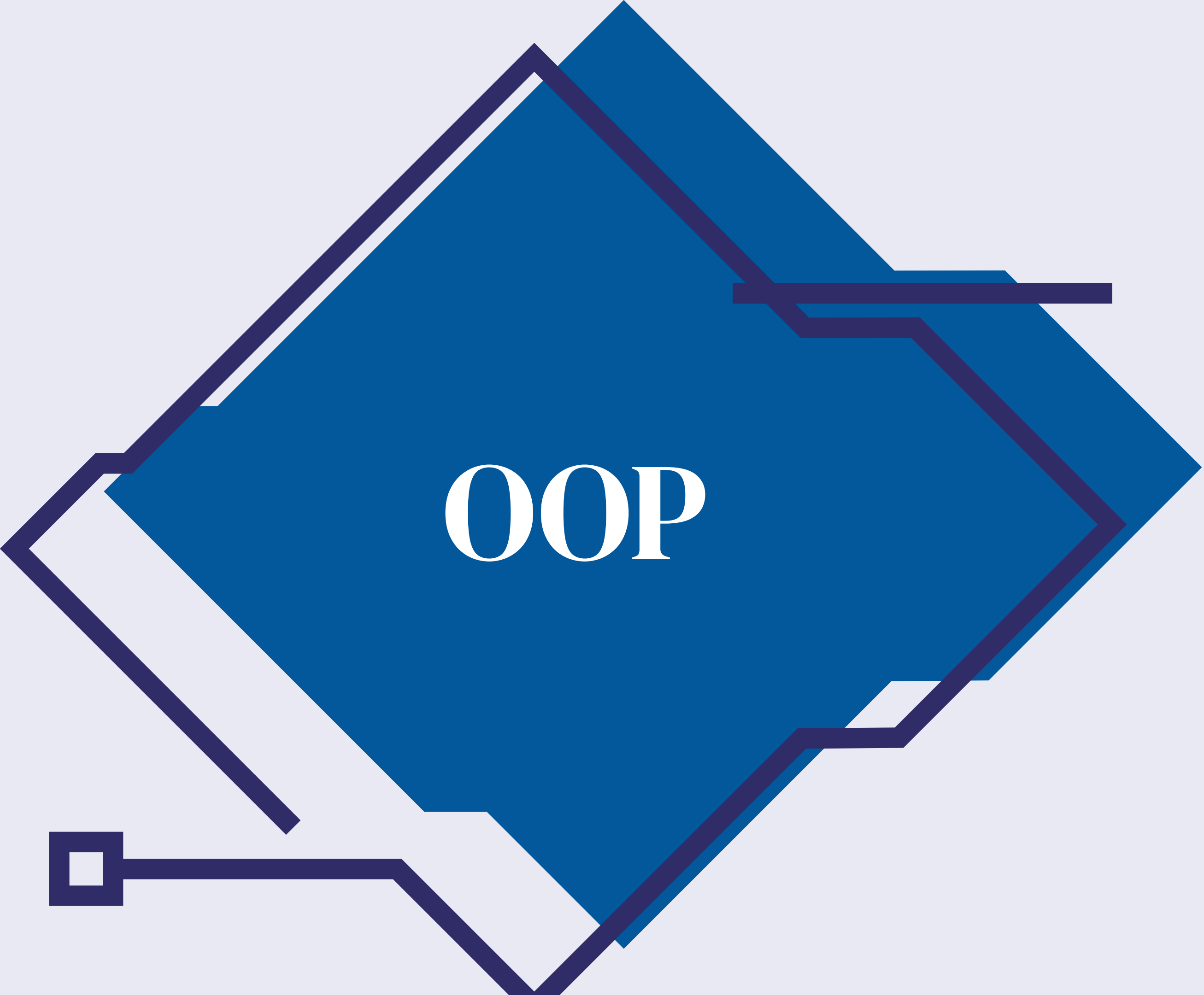
Look at this code without running it first — predict what happens (Error? Specific output? Which line fails?):

```
● ● ●  
  
int total = 100;  
  
void applyDiscount() {  
    int discount = 10;  
    total = total - discount;  
    print('Inside function: $total');  
}  
  
void main() {  
    applyDiscount();  
    print('Outside function: $total');  
    print('Discount was: $discount');  
}
```





10 min Break

An abstract geometric composition on a light blue background. A large, solid blue diamond is the central element. Overlaid on and around this diamond are several dark blue lines of varying thickness. Some lines form a jagged, stepped border around the diamond, while others are more fragmented and extend outwards. In the bottom-left corner, there is a small square containing a white square, with a line extending from it towards the main diamond structure.

OOP

What is a Programming Paradigm?

A paradigm is simply a way of thinking about how to structure and write code — a general approach or philosophy, not a specific language or tool.

Common Paradigms:

Object-Oriented Programming (OOP)

Procedural Programming

Functional Programming

Why OOP is important

Models the real world accurately – data and behavior that naturally belong together stay together (an account's balance and its deposit/withdraw actions)

Reusability – one class = a blueprint for unlimited objects (1 account, 1,000 accounts — same code)

Organization at scale – large programs stay manageable because related code is grouped, not scattered

Data protection – an object can control how its own data is accessed and changed (more on this in Encapsulation)

Independence – each object has its own copy of data; changing one object never affects another

class

blueprint — a definition of what properties and behaviors something should have. It's not a real thing yet, just the design.



```
class Patient {  
    // Properties (Attributes)  
    String name;  
    int age;  
    String bloodType;  
  
    // Constructor  
    Patient(this.name, this.age,  
this.bloodType);  
  
    // Methods (Behaviors)  
    void bookAppointment() {  
        print("$name booked an  
appointment.");  
    }  
  
    void displayInfo() {  
        print("Name: $name");  
        print("Age: $age");  
        print("Blood Type: $bloodType");  
    }  
}
```

What is an Object

- Class = Blueprint
- Object = Real Instance
- We interact with Objects, not with Classes



```
void main() {  
    Patient patient1 = Patient("Ahmed",  
25, "A+");  
    Patient patient2 = Patient("Mona", 30,  
"O+");  
    Patient patient3 = Patient("Ali", 40,  
"B-");  
}
```

Creating an Object

The object should receive its data as soon as it is created



```
Patient patient1 = Patient();
```



```
patient1.name = "Ahmed";  
patient1.age = 25;  
patient1.bloodType = "A+";
```

What is a Constructor

- A Constructor is a special method.
- It is called automatically when an object is created.
- It has the same name as the class.
- It is used to initialize the object's data.



```
class Patient {  
    String name;  
  
    Patient(this.name);  
}
```

Default Constructor

A constructor without parameters.



```
class Patient {  
    String name;  
  
    Patient(this.name);  
}
```

Parameterized Constructor

Receives values while creating the object.



```
class Patient {  
    String name;  
    int age;  
  
    Patient(this.name, this.age);  
}
```



```
Patient patient1 = Patient("Ahmed", 25);
```

Named Constructor

Provides multiple ways to create an object.



```
Patient patient = Patient(  
    name: "Ahmed",  
    age: 25,  
);
```



```
class Patient {  
    String name;  
    int age;  
  
    Patient({  
        required this.name,  
        required this.age,  
    });  
}
```

Constructor with Named Parameters

a collection of key-value pairs (like a real dictionary: word → meaning)



```
Map<String, int> students = {"ahmed": 6,  
"nada": 10};
```

```
Map userInfo = {  
    "userName": "ali123",  
    "age": 16,  
    "country": "egypt",  
};
```

```
print(userInfo.keys);    // (userName,  
age, country)  
print(userInfo.values);  // (ali123, 16,  
egypt)  
print(userInfo["age"]);  // 16  
print(userInfo.length);  // 3
```

Destructor

A Destructor is a special method that is called automatically when an object is destroyed or removed from memory.

Releases allocated resources.

Cleans up memory.

Executes before the object is destroyed.



```
class Patient {  
public:  
    ~Patient() {  
        cout << "Patient Destroyed";  
    }  
};
```

Instead, Dart uses a Garbage Collector (GC).

- Automatically detects objects that are no longer used.
- Frees memory automatically.
- Developers don't need to destroy objects manually.

Why Garbage Collection?

- Prevents memory leaks.
- Makes memory management easier.
- Improves application stability.